

ORION: Intent-Aware Orchestration in Open RAN for SLA-Driven Network Management

Gabriela da Silva Machado, Gustavo Z. Bruno[✉], Alexandre Huff[✉], and Cristiano B. Both[✉]

Abstract—

Index Terms—Intent-Based Networking, Open RAN, O-RAN, Network Automation, SLA Assurance, Radio Resource Management.

I. INTRODUCTION

The Fifth Generation (5G) mobile communications ecosystem has embraced disaggregation and openness to unlock flexible service delivery, yet these same properties amplify the operational burden on Radio Access Network (RAN) operators [1]. The Open Radio Access Network (O-RAN) architecture formalizes this transition by splitting control across the Service Management and Orchestration (SMO) framework and the Non-Real-Time (Non-RT) and Near-Real-Time (Near-RT) RAN Intelligent Controllers (RICs), each exposing programmatic interfaces for policy-driven control. In practice, operators must coordinate a growing catalog of Non-real-time RIC applications (rApps) and Near-real-time RIC applications (xApps) and reconcile the pace of software updates with strict Service Level Agreements (SLAs) and Service Level Objectives (SLOs). They must also maintain end-to-end observability across disaggregated components [2].

Intent-Based Network (IBN) has emerged to bridge high-level business goals and low-level network actions by offering formalism to capture desired outcomes and automating the translation into enforceable policies [3], [4]. Telecom bodies such as 3rd Generation Partnership Project (3GPP) and European Telecommunications Standards Institute (ETSI) now specify intent-driven management hooks for RAN automation. Industry proposals extend IBN with data-driven and Large Language Models (LLMs) assistants [5], [6], [7]. However, current IBN efforts focus either on cloud-native infrastructure abstractions or on intent-to-policy translation detached from O-RAN's dual-controller pipeline, delivering limited guarantees on SLA preservation and lacking an integrated lifecycle that spans clarification, validation, deployment, and continuous adaptation [8], [9], [10].

These gaps motivate our investigation of intent-aware orchestration tailored to O-RAN. We study how to couple LLM-assisted interfaces with O-RAN-compliant control loops so

that operator intents become deployable, monitorable policies without compromising latency, throughput, or reliability commitments. The research question guiding this work is: *How can O-RAN management be enabled using intents without compromising network quality and SLA compliance?*

This paper makes the following contributions:

- We introduce ORION, an intent-aware orchestration pipeline that aligns LLM-assisted intent capture with the SMO, Non-RT RIC, and Near-RT RIC roles, structuring slice-related requests through CAMARA's *NetworkSlice-Booking* schema to ground translations in telecom semantics.
- We design conflict-aware validation and composition flows that span SMO applications, rApps, and xApps, ensuring intents progress through schema checks, capability matching, and policy generation before enforcement via A1 Interface (A1) and E2 Interface (E2) interfaces while sustaining Closed-Loop Assurance (CLA) loops.
- We implement a prototype combining Model Context Protocol (MCP) services, policy catalog adapters, and O-RAN-compliant control functions that operationalizes the proposed orchestration pipeline.

We evaluate ORION by examining translation fidelity, orchestration latency, and SLAs/SLOs adherence for representative slicing scenarios.

The remainder of the paper is organized as follows. Section II reviews O-RAN orchestration and intent-based networking foundations. Section III surveys prior art on intent-driven RAN management. Section IV details the ORION architecture, and Section V summarizes the prototype. Section VI presents the evaluation and experimental analysis, while Section VII concludes the paper.

II. BACKGROUND

A. O-RAN Architectural Elements

O-RAN disaggregates the 3GPP gNodeB (gNB) into virtualized Central Unit (CU) and Distributed Unit (DU) plus Radio Unit (RU) [1], enabling multi-vendor deployment and software-driven control. Two intelligent controllers, the Non-RT RIC (within the SMO) and the Near-RT RIC (edge), enable policy- and Machine Learning (ML)-driven optimization at complementary time scales (Non-RT: > 1 s; Near-RT: 10 ms–1 s) [11]. The SMO hosts modular management functions and rApps (service-level analytics and optimization), while the Near-RT RIC hosts xApps that execute near-real-time control loops over radio resources.

Data and control traverse distinct open interfaces: A1 conveys guidance (policies, enrichment data, feedback) from the

Manuscript received April XX, 2025. The research leading to this paper received support from the Program OpenRAN@Brasil. This work also received support from the Commonwealth Cyber Initiative, an investment in the advancement of cyber R&D, innovation, and workforce development.

Gustavo Zanatta Bruno is with the National Telecommunications Institute (INATEL). E-mail: gustavo.zanatta@posdoc.inatel.br.

Gabriela da Silva Machado and Cristiano B. Both are with the University of Vale do Rio dos Sinos (UNISINOS). E-mail: cbboth@unisinos.br.

Alexandre Huff is with the Federal Technological University of Paraná (UFTPR). E-mail: alexandrehuff@utfpr.edu.br.

Non-RT RIC to the Near-RT RIC; E2 streams telemetry (e.g., cell load, Physical Resource Block (PRB) usage) upward and distributes fine-grained control actions downward to CU/DU; and O1 Interface (O1) supports Fault, Configuration, Accounting, Performance, Security (FCAPS) management between the SMO and RAN elements (inventory, configuration, faults, performance counters) [1]. This layering separates concerns: long-horizon optimization and model training in the Non-RT domain, and sub-second adaptation and enforcement in the Near-RT edge domain.

The openness that enables innovation also creates coordination challenges: (i) heterogeneous vendor data models must be normalized; (ii) timing alignment between A1 (seconds) and E2 (milliseconds) control loops must avoid oscillations; and (iii) intent translation must ensure consistency when multiple rApps/xApps manipulate overlapping Key Performance Indicators (KPIs) [8]. These issues motivate the structured abstractions (intents, policies, roles) described next.

B. Policies vs. Intents

In O-RAN, policies are structured rules: when events and conditions hold, specified actions execute to steer performance (e.g., load balancing or Quality of Service (QoS) objectives) across scoped entities (cells, slices, User Equipment (UE) groups) via A1-defined JavaScript Object Notation (JSON) policy types [1]. They express “if/when → do” logic without explicitly declaring end-state semantics. Intents, by contrast, declare desired outcomes or operational goals (the *what*), omitting the procedural *how* [5]. A single intent may decompose into multiple policies across domains (optimization, assurance, resource allocation) [3].

Abstraction boundary: policies bind directly to enforcement interfaces (A1 toward the Near-RT RIC, O1 for management operations, and, indirectly, E2 control via xApps) [1], whereas intents act as a contract specifying KPIs (e.g., “average cell-edge throughput $\geq X$ Mbps within latency constraint Y ”) [5]. Translation engines map such KPI-oriented statements into one or more concrete policy objects plus monitoring queries. Policies are evaluated reactively on triggers; intents are continuously assured, with fulfillment engines monitoring KPIs against target states and adapting underlying policies [12].

Challenges include conflict detection (e.g., two intents driving contradictory PRB allocation goals), measurability (ensuring each intent has observable success metrics), and drift handling (environmental changes invalidating earlier translations) [3]. Effective frameworks therefore (i) capture goals and constraints, (ii) validate feasibility against current topology and resources, (iii) translate into minimal, consistent policy sets, and (iv) maintain CLA adjustments to preserve declared outcomes [5], [12].

C. Intent Lifecycle

The lifecycle comprises: Create (register goals, scope, constraints, reporting), Activate (begin fulfillment), Monitor/Assure (observe KPIs, detect deviation), Modify (adjust goals/constraints or suspend state), Suspend (temporarily halt

fulfillment), and Terminate (cease monitoring, clean up). Each state transition is auditable and can emit events (e.g., activation success, assurance failure) that feed higher-level orchestration.

CLA evaluates whether observed state matches the declared outcome; deviations trigger re-translation (e.g., updated A1 policies) or operator notification, per 3GPP TS 28.312 [5]. Practical considerations include hysteresis (to avoid flapping), rollback (to restore the last stable policy set), and lifecycle coupling (terminating a parent intent cascades termination to derived optimization intents) [12]. Structured lifecycle handling reduces ambiguity and enables automated recovery and adaptation.

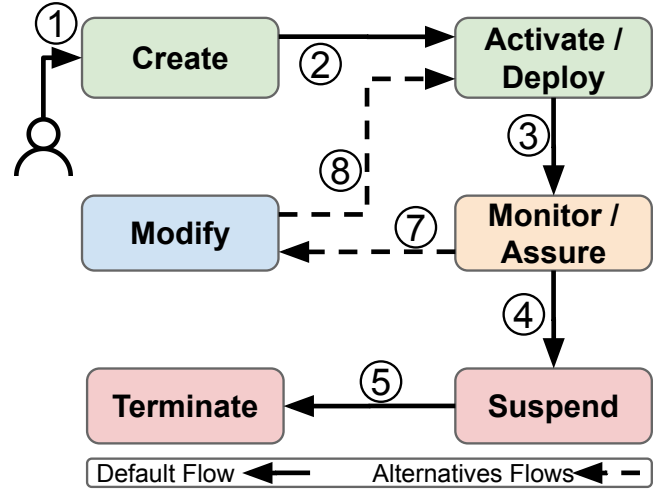


Fig. 1. Intent lifecycle states and transitions with default flow (1–6) and CLA feedback loops (7–8).

Fig. 1 illustrates the lifecycle states and transitions. Solid arrows (1–6) depict the nominal flow from creation through activation and assurance to termination. Dashed arrows highlight the CLA feedback loops essential for O-RAN: arrow 7 represents corrective actions triggered when KPIs deviate during monitoring, while arrow 8 captures re-translation or rollback following modifications. This adaptive cycle ensures continuous alignment with declared goals under dynamic network conditions.

D. Roles in Intent-Driven RAN

Within SMO-based handling, the RAN Management Intent Owner (RMIO) formulates intents (goals, constraints, KPI targets, reporting cadence), owns lifecycle authority (modify, suspend, terminate), and defines assurance requirements. The RAN Management Intent Handler (RMIH) advertises supported intent classes, interprets RMIO submissions, decomposes them into actionable policies/configurations (A1 guidance, O1 management actions), orchestrates deployment, resolves alternative fulfillment strategies (optionally consulting the RMIO), and performs continuous assurance.

Interaction flow: the RMIO queries capability advertisements, submits a validated intent, receives an identifier and reporting schedule, and is asynchronously notified of assurance status or required decisions (e.g., selecting among

alternative resource trade-offs). The RMIH maintains a dependency graph mapping intents to generated policies and underlying xApp/rApp actions, enabling targeted updates rather than wholesale recomputation when only one KPI threshold changes.

The RMIH manages conflicts across coexisting intents (priority-based arbitration or multi-objective optimization) [3], adapts policies upon KPI drift, and issues status/fulfillment reports [12]. This separation clarifies accountability (ownership vs. operational realization) and supports scalable multi-tenant automation in O-RAN, while providing clear hooks for auditing, rollback, and intent evolution.

Despite architectural openness, consistent and explainable translation from high-level intents to coherent multi-interface policies (A1, O1, E2), while assuring outcomes under dynamic traffic, remains non-trivial. This motivates the related work and the proposed architecture that follow.

III. RELATED WORK

This section surveys intent-driven management for RAN/O-RAN, focusing on five aspects that shape practical systems: (i) how intents are represented, (ii) how they are translated into enforceable actions, (iii) lifecycle coverage and assurance, (iv) scope and use in slicing, and (v) alignment with the O-RAN framework. We synthesize observations from recent proposals [12], [3], [13], [8], [14], [15], [16], [17], [18] and standards guidance [5], [19], [20], [4]. Table I summarizes the surveyed works across intent representation, translation, lifecycle/assurance, slicing, and O-RAN alignment.

Standards landscape. The treatment of intent has matured across Standards Development Organizations (SDOs). TM Forum (TMF)’s TMF921 Intent Management API (v5.0.0) binds explicitly to the TMF Intent Ontology, clarifying validation, capabilities, and exchange of machine-tractable intent objects [21], [20]. In 3GPP, TS 28.312 (Rel-18 v18.8.0) formalizes quantifiable intents, feasibility and fulfillment reports, and priority/conflict semantics; it also details discovery of handler capabilities and aligns SMO/Non-RT roles with lifecycle hooks [5], [22]. ETSI Zero touch network and Service Management (ZSM) provides the reference model for owner/handler roles and closed-loop operations [19], and ZSM 016 adds a three-level conflict taxonomy with concrete detect-report-judge-fulfill interactions [6]. From the O-RAN angle, ETSI TS 103 985 (PAS) captures O-RAN WG2 A1 policy/Enrichment Information (EI) jobs and status flows [23], while Request for Comments (RFC) 9315 standardizes IBN terminology and separates fulfillment from assurance [4]. Together, these specifications define the contract for intent handling but stop short of an end-to-end, O-RAN-native reference instantiation.

Intent representation. Prior work spans natural-language interfaces, structured templates, and standards-based models. Natural-language inputs with operator-facing portals appear in [17], [18], [8], often coupled to guided forms to constrain ambiguity (e.g., specifying a desired KPI improvement like “increase throughput by 10%” [8]). Template- or rule-oriented forms are used in [12], [3], where operators describe goals in

predefined fields that map to policy rules. A standards-aligned approach adopts the TMF intent model, technology-agnostic goals with explicit semantics, as in [16], [13], which reduces interpretation ambiguity and eases decomposition into policies.

Translation and interpretation. Mapping high-level goals to concrete policies/configurations is handled via rule pipelines, graph-based reasoning, or Natural Language Processing (NLP)/Artificial Intelligence (AI)-assisted engines. [12] validates admissibility (KPI/control-parameter scope), classifies content, then emits commands. Graph-based translators in [15], [14] consult topology/resource/policy repositories to build intent–resource graphs and generate domain-specific RAN policies. [17] separates an Intent Engine (NLP interface, Application Programming Interface (API) intent matching, interpretation to API calls) from an AI Engine hosting ML models for low-level context, while TMF-aligned handling in [16] follows capability-driven translation. Several works mention validation/translation steps without full detail [3]. Recently, LLMs have been placed directly in the control loop: LLM-xApp translates dynamic UE QoS intents into slice configurations at Near-RT timescales [10]; LLM-hRIC organizes translation hierarchically with LLM-guided Non-RT guidance and Reinforcement Learning (RL)-based Near-RT enforcement [24]; Orange proposes FEACI to score translation quality across RAN+5G Core (5GC) [9]; and INA-Infra demonstrates an open O-RAN-based stack (Nephio+rApp+xApp) that operationalizes intent-to-slice flows [7]. Overall, a common pattern is feasibility checking, capability matching, construction of minimal consistent policy sets, and continuous feedback.

Lifecycle, assurance. Lifecycle operations (create, modify, suspend, terminate) and handler interfaces appear in [16], while [18] maintains CLA by monitoring performance and adapting policies. These notions align with ETSI ZSM’s owner/handler roles and declarative outcomes [19], 3GPP TS 28.312’s intent semantics and lifecycle hooks [5], and RFC 9315’s distinction between fulfillment and assurance with drift remediation [4]. Standards have strengthened intent semantics and instrumentation: TMF921 v5.0.0 (Oct. 2024) binds to the TMF Intent Ontology (TIO), clarifying validation and exchange [21]; 3GPP TS 28.312 Rel-18 v18.8.0 (Jul. 2025) formalizes quantifiable intents, priority/conflict handling, feasibility, and report subscription for SMO/Non-RT handlers [5], [22]. Research on runtime assurance and conflicts includes RSLAQ’s SLA→KPI mapping with A1/E2 monitoring [25] and PACIFISTA’s rApp/xApp profiling with graph reasoning to predict conflict severity and Key Performance Metric (KPM) impact [26]. Many prototypes still emphasize ingestion/initial fulfillment over long-horizon assurance, conflict reconciliation, and rollback.

Alignment with O-RAN. Only a subset explicitly instantiates O-RAN’s Non-RT/Near-RT split with open interfaces. [3] places long-term policy/ML in the Non-RT RIC and real-time control in the Near-RT RIC via A1/O1, while [8] uses hierarchical Deep Reinforcement Learning (DRL) with Non-RT rApps issuing goals over A1 to Near-RT xApps. [17] integrates an intelligence layer adjacent to RAN, borrowing O-RAN concepts without a native end-to-end loop. Newer materials underscore native treatment of intent and conflicts

in Non-RT/Near-RT loops, ETSI TS 103 985 captures A1 policy/EI jobs and status [23], and the O-RAN Alliance published an SMO Intent-driven Management TR (v5.0) and an A1 Policy Conflict Mitigation Study (v1.00) [27]. RL-based, O-RAN-consistent enforcement pipelines also appear (AdapShare; AI/ML-driven SMO) [28], [2].

Slicing scope and use cases. Objectives range from KPI/control-parameter tuning [12] to performance steering via rApp/xApp orchestration [3], [8], with slicing-focused frameworks optimizing SLA satisfaction and resource allocation across RAN and sometimes 5GC [14], [15], [16], [13]. Private 5G management with intent-driven slice provisioning is explored in [17], while [18] maps intents to slice instances in a power-grid automation context.

Across the literature, there is convergence on: (i) declarative goal capture with explicit KPIs and scope; (ii) repository-backed translation (capabilities, topology, policies) to derive consistent policies; and (iii) multi-timescale control where Non-RT guidance informs Near-RT enforcement. Yet no work delivers a *complete* ingestion→translation→enforcement→CLA pipeline that is *natively O-RAN-embedded* (A1/O1/E2), *standards-aligned* (TMF & 3GPP models), and provides *first-class conflict management* (priority, reconciliation, degradation); even the most advanced proposals (e.g., RSLAQ’s SLA-to-KPI loop or LLM-xApp’s translator) cover only slices of the pipeline, and standards (ZSM 016, TS 28.312) specify roles and messages but not an end-to-end reference instantiation. The proposed architecture addresses this gap with an ingestion-to-enforcement pipeline that exposes capability discovery, aligns lifecycle management with standards, and sustains CLA over O-RAN interfaces.

IV. PROPOSED ARCHITECTURE: ORION

This section presents ORION, an intent-aware orchestration pipeline for O-RAN that bridges natural-language intents and deployable RAN policies. ORION couples an LLM-assisted MCP client/server with O-RAN-compliant control functions (Non-RT/SMO and Near-RT RIC) and reuses the CAMARA *NetworkSliceBooking* API schema as a canonical template to structure slice-related intents.

A. Design Objectives

ORION targets five objectives. First, automation with SLA-awareness so that intents are translated into machine-validated specifications that preserve latency, throughput, and reliability KPIs. Second, modularity that cleanly decouples intent assistance, interpretation, composition, and policy handling, enabling each block to evolve independently. Third, alignment with existing roles and interfaces by mapping responsibilities to SMO/RIC, adopting RMIO/RMIH semantics, and reusing open API schemas such as CAMARA. Fourth, strict O-RAN compliance by interacting with the Non-RT RIC for A1 policy distribution and the Near-RT RIC/xApps for E2 enforcement. Fifth, extensibility beyond slicing (e.g., energy saving and congestion mitigation) by swapping schemas and tool adapters without redesigning the pipeline.

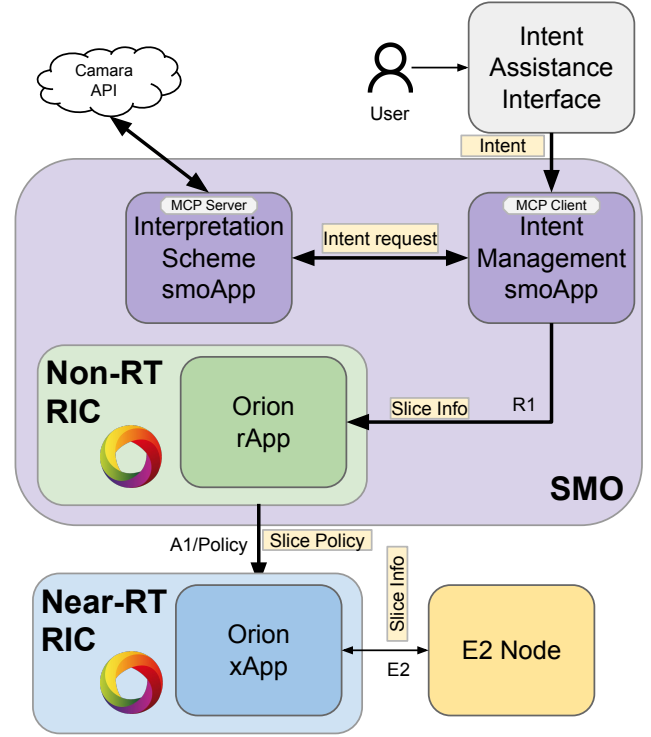


Fig. 2. ORION high-level architecture spanning SMO/Non-RT and Near-RT RIC: the MCP Client/Server structure intents using CAMARA’s *NetworkSliceBooking* schema; the Orion rApp composes policies, and the Orion xApp enforces them via A1/E2.

B. High-Level Architecture Overview

Figure 2 depicts ORION’s blocks spanning the SMO/Non-RT and Near-RT layers. The Intent Assistance Interface (IAI) (MCP Client) detects intents from an operator or higher-layer system, invokes an LLM with domain tools, and mediates between free text and structured forms. The MCP Server provides tool endpoints and context to the LLM, including adapters to external telecom APIs; for slicing, it exposes a translator aligned to CAMARA *NetworkSliceBooking*. Within the SMO, the Intent Management (smoApp) tracks the lifecycle, validates structured intents, and passes accepted ones to composition. The Interpretation Scheme Module (ISM) handles ontology and schema validation, checking completeness and admissibility against policy catalogs and network capabilities. The Orion rApp (Orion rApp) converts structured intents into concrete policy templates and parameters, performing domain calculations (e.g., mapping latency budgets to PRB allocations, scheduling choices, and QoS classes). Finally, the Orion xApp (Orion xApp) operates with A1/E2 interaction: the Non-RT RIC publishes policies via A1 to the Near-RT RIC, and the Orion xApp enforces or refines control through E2 toward gNBs (DU/CU).

C. Component Details

The MCP Client and Server jointly realize the intent assistance plane. The client orchestrates the LLM workflow: it recognizes incoming intent requests, forwards them to the server, invokes the appropriate tool, and returns the structured

TABLE I
RELATED WORK ON INTENT FOR RAN/O-RAN: REPRESENTATION, TRANSLATION, LIFECYCLE/ASSURANCE, SLICING, AND O-RAN ALIGNMENT

Year		Paper	Intent Rep.	Translation	Lifecycle/Assurance	Slicing	O-RAN Align.
2020	[15]	Graph-based RAN slice orchestration	●	●	○	●	○
2021	[12]	Rule-pipeline intent to KPI commands	●	●	○	○	○
2021	[3]	End-to-end IBN (standardized model)	●	●	○	○	●
2021	[18]	Intent-driven slice mgmt (power grid)	●	●	●	●	○
2022	[16]	TMF-aligned intent for E2E slicing	●	●	●	●	●
2023	[17]	NLP-powered private 5G management	●	●	○	●	○
2023	[8]	Hierarchical DRL for O-RAN xApp orchestration	●	●	○	●	●
2023	[14]	Intent-driven RAN slice orchestration (MARL)	●	●	○	●	○
2024	[13]	Intent-aware RRS for RAN slicing (RL)	●	●	○	●	○
2024	[2]	AI/ML-driven SMO framework in O-RAN	●	●	○	●	●
2024	[7]	INA-Infra: open intent-driven slicing stack	●	●	●	●	●
2024	[28]	AdapShare: RL-based intent spectrum sharing	○	●	○	●	●
2024	[26]	PACIFISTA: O-RAN conflict characterization	○	●	●	○	●
2025	[24]	LLM-hRIC: LLM-guided Non-RT + RL Near-RT	●	●	○	●	●
2025	[10]	LLM-xApp: NL-to-slice translation at Near-RT	●	●	○	●	●
2025	[9]	FEACI: translation quality scoring (RAN+Core)	●	●	○	●	○
2025	[25]	RSLAQ: SLA-driven QoS with A1/E2 loops	●	●	●	●	●

● Yes ● Partial ○ No

outcome to the SMO. It maintains conversation state and applies guardrails that promote deterministic mappings from natural language to schema fields. The server exposes tool endpoints and domain context to the LLM. For slicing, the server supplies a translator aligned with CAMARA’s *NetworkSliceBooking* API¹. This translator accepts an intent payload, builds a compliant JSON post, optionally consults a slice catalog, and returns a structured result. The design grounds the LLM in telecom constraints and decouples credentials and API logic from the model prompt. ORION adopts the *NetworkSliceBooking* resource model (e.g., slice profile, QoS requirements, area of service, duration) as a canonical skeleton for slice intents, enabling vendor-agnostic translation and simplifying validation and auditing.

Within the SMO/Non-RT domain, the Intent Management (smoApp) registers, versions, and validates intents. It triggers the ISM, which performs ontology and schema checks for completeness (mandatory fields), consistency (ranges and units), and feasibility (capability match and policy conflicts). Ambiguities are surfaced to the MCP Client to request clarifications, ensuring that only admissible, complete intents progress to composition.

The Orion rApp and the Orion xApp implement translation-to-enforcement. Given a validated structured intent, the Orion rApp computes control parameters and maps them into policy templates; for slicing, this includes allocating throughput/latency/reliability budgets, selecting scheduler and QoS classes, and defining E2 monitoring metrics. The Orion rApp annotates priorities and dependencies used later for arbitration. Policies are then published via A1 by the Non-RT RIC and enforced by the Orion xApp through E2 towards gNBs (DU/CU). The Orion xApp monitors KPMs and drives CLA updates by requesting recomputation or rollback when deviations are detected.

D. Workflow: Intent Lifecycle

The end-to-end lifecycle covers six phases.² Ingestion: an operator issues a slice intent and the MCP Client initializes the LLM session. Translation: the LLM calls the MCP Server tool, which structures the request per CAMARA *NetworkSliceBooking* and returns JSON. Validation: the smoApp and Interpretation module verify correctness and feasibility and may request clarifications. Composition: the rApp turns the validated intent into policy templates and parameter values. Deployment: the Non-RT RIC publishes policies via A1 and the xApp enforces them through E2 toward gNBs. Monitoring and adaptation: KPIs trigger recomputation, incremental updates, or rollback when deviations appear.

E. Conflict Handling and Resolution

ORION applies priority-based arbitration and dependency checks during composition and deployment. Intent priority and scope determine precedence. Semantic conflicts (for example, overlapping PRB reservations) are detected through capability and inventory queries. If a conflict remains unresolved, a safe rollback reverts to the last consistent policy set while the MCP Client requests clarification. The rApp emits a conflict report with impacted cells, parameters, and suggested mitigations.

F. Extensibility and Standards Alignment

Roles map to RMIO/RMIH: the operator or service assumes RMIO, while the smoApp+rApp act as RMIH. ORION aligns with O-RAN by placing intent management and composition in the Non-RT/SMO domain and using A1/E2 for control. By treating CAMARA APIs as interchangeable schema backends, the same pattern generalizes to other intents (energy saving, load balancing) by swapping the tool and policy templates without altering the core workflow.

¹<https://github.com/camaraproject/NetworkSliceBooking>

²For brevity, IAI and MCP Client are used interchangeably.

Example trace.: "Provision a URLLC slice in area X with 1 ms latency and 99.999% reliability for 2 hours." An operator issues the request, the MCP Client opens the LLM session, and the MCP Server tool produces NetworkSliceBooking JSON with latencyTarget=1 ms, reliability=99.999%, area=X, and duration=2 h. The smoApp validates the payload and forwards it to the rApp, which derives scheduler priorities and PRB reservations before emitting an A1 policy. The xApp enforces the policy through E2 and monitors URLLC KPIs; if latency exceeds 1 ms, the rApp tightens scheduling deadlines, scales resources, or triggers rollback to a previous stable configuration.

V. IMPLEMENTATION AND PROTOTYPE

This section describes the implementation of the Near-RT RIC components developed for the network slicing control loop. The architecture consists of the Orion xApp (*orion-xapp*) and an E2 Node simulator (*e2sim-rc*) that communicate via the O-RAN E2 interface.

A. Component Architecture

1) *Orion xApp*: The *orion-xapp* is a C++ based application deployed on the Near-RT RIC platform that serves as the central control entity responsible for translating high-level intents into low-level RAN configurations. Acting as the A1 Interface termination point, it receives JSON-based policies from the Non-RT RIC via the A1 Mediator. Upon receiving a policy, the Orion xApp queries the E2 Manager (E2Mgr) to discover connected E2 nodes and their capabilities. The core functionality involves a protocol translation engine that maps abstract slicing objectives, such as throughput requirements, into E2SM-RC (E2 Service Model for RAN Control) structures. These control messages are then encoded using the Abstract Syntax Notation One (ASN.1) Packed Encoding Rules (PER) for transmission over the E2AP protocol.

2) *E2 Simulator (e2sim-rc)*: The *e2sim-rc* is an extension of the O-RAN Software Community (OSC) E2 Simulator, modified to support E2SM-RC, emulating a gNB capable of advanced resource control. During the initialization phase, it performs the E2 Setup procedure, advertising support for E2SM-RC Control Style 2 (Radio Resource Allocation) and Action ID 6 (Slice-level PRB quota). The simulator is designed to decode incoming RIC Control Requests and apply PRB partitioning logic. Additionally, it implements a capacity validation mechanism acting as a resource guard to reject any control requests that would exceed the total available PRB capacity of the cell, ensuring network stability.

B. Near RT RIC Workflow

The interaction between the Orion xApp and the RAN is illustrated in the sequence diagram (Figure 3). The process follows a twelve-step control loop triggered by the arrival of a new policy.

The cycle begins when the A1 Mediator pushes a policy to the Orion xApp via the RMR interface (**Step 1**). The Orion xApp parses the JSON payload (**Step 2**) to extract

the target slice identifier and SLA objectives. To determine enforcement targets, the Orion xApp queries the E2 Manager (**Step 3**), which returns a list of connected E2 nodes and their inventory (**Step 4**). The Orion xApp's translation engine then converts the parsed SLA parameters into an E2SM-RC Control Header and Control Message (**Step 5**), specifying the Control Style (Type 2), Action ID (6), and RAN Parameters for PRB Policy Ratios. These structures are encapsulated into an E2AP RICcontrolRequest PDU and encoded into binary format (**Step 6**).

The encoded message is sent to the E2 Termination via RMR (**Step 7**), which forwards it to the specific E2 Node over SCTP (**Step 8**). Upon receipt, the *e2sim-rc* decodes the payload and processes the request (**Step 9**), validating that the new allocation respects the node's capacity before updating the scheduler. If successful, the simulator generates a RICcontrolAcknowledge message sent back to the E2 Termination (**Step 10**). This acknowledgement is routed to the Orion xApp (**Step 11**), confirming the configuration. Finally, the Orion xApp reports the operation status to the Non-RT RIC via an A1 Policy Response (**Step 12**), closing the control loop.

VI. EVALUATION

A. Resource Consumption Analysis

We assessed the computational overhead of the proposed intent orchestration framework using the Prometheus monitoring stack to scrape CPU and memory metrics from the target Kubernetes namespaces. The evaluation captured the steady-state performance of the O-RAN platforms (Non-RT RIC, Near-RT RIC) and the developed MCP Server/Client, alongside the specific intent-handling components (Orion rApp, Orion xApp) and the E2 simulator. As illustrated in Figure 4, the resource distribution reveals a stark contrast between the baseline infrastructure platforms and the lightweight orchestration logic implemented in this work.

The quantitative results confirm the efficiency of our approach. The Non-RT RIC remains the most resource-intensive component, consuming an average of 76.4 ± 3.4 m CPU and ≈ 7.1 GiB of memory due to its comprehensive microservice suite, followed by the Near-RT RIC at 37.1 ± 4.6 m CPU and 1.1 GiB memory. In comparison, the custom intent applications impose negligible overhead: the Orion rApp and Orion xApp consume only 1.5 ± 0.3 m and 0.4 ± 0.0 m CPU respectively, with memory footprints under 40 MiB. With a total system-wide draw of approximately 141 millicores and 8.5 GiB RAM, the solution proves viable for deployment on resource-constrained edge servers.

B. End-to-End Processing Time Analysis

To evaluate the performance of the proposed intent-driven slicing framework, we conducted an end-to-end processing time analysis measuring the latency introduced by each architectural layer. Processing times were calculated by extracting timestamped events from Kubernetes pod logs across all

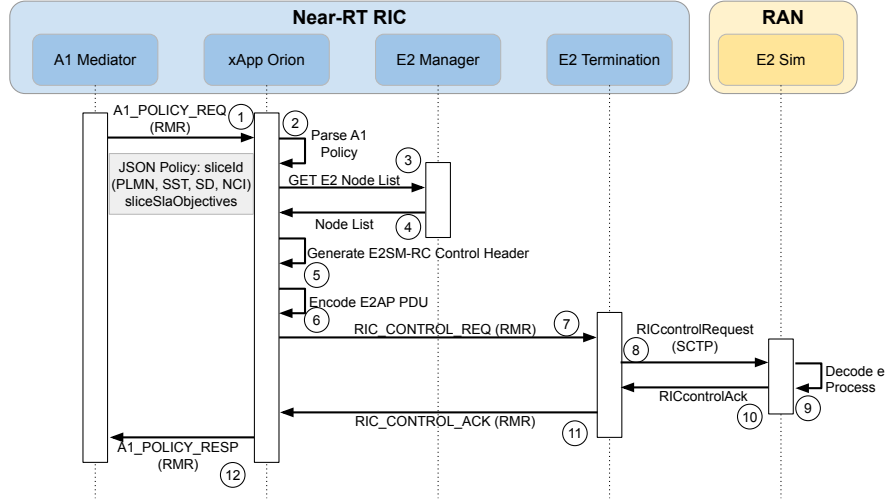
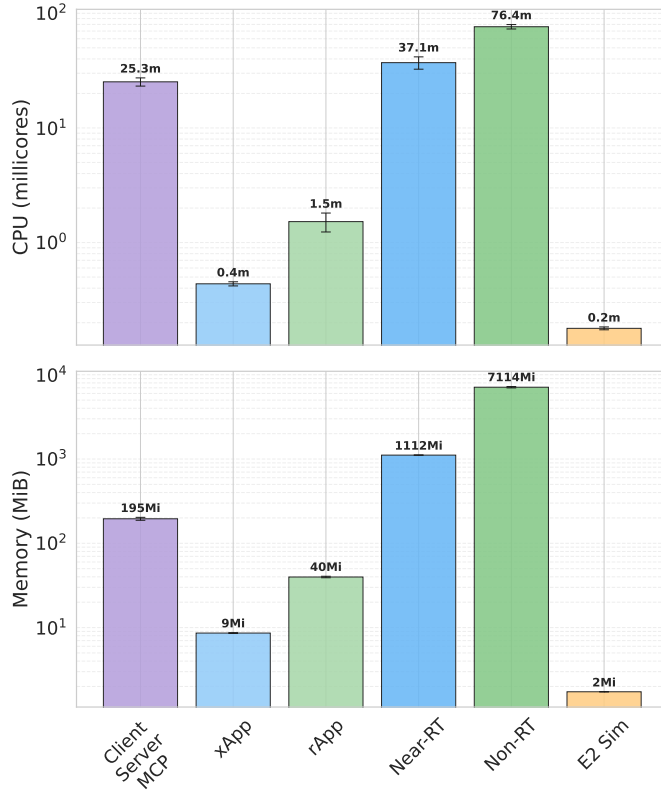


Fig. 3. Sequence diagram of the control loop.

Fig. 4. CPU (millicores) and Memory (MiB) usage by namespace (Mean $\pm$ Std Dev).

framework components, correlating event pairs using policy instance identifiers or time-proximity matching. We instrumented key events at the SMO layer (intent reception, LLM processing, MCP tool execution, and A1 policy creation), the **Non-RT RIC!** (**Non-RT RIC!**) layer involving the Orion rApp (policy translation and PUT request completion), the **Near-RT RIC!** (**Near-RT RIC!**) layer via the Orion xApp (A1 Mediator forwarding, policy parsing, E2 Service Model

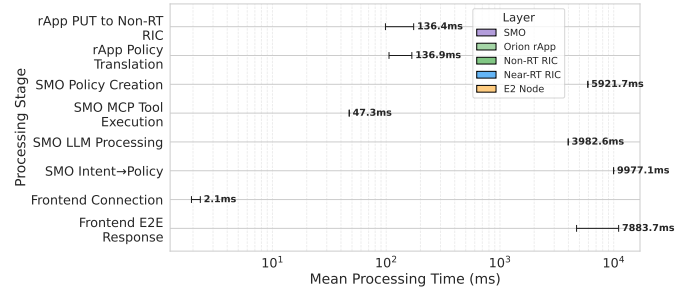


Fig. 5. Mean processing time by stage across the intent-driven slicing framework. Error bars indicate standard deviation. Note the logarithmic scale on the x-axis.

RAN Control (E2SM-RC) message encoding), and finally the E2 Node (control reception, slice registration, and acknowledgment transmission).

Figure 5 presents the mean processing time for each stage, revealing that the SMO layer dominates the end-to-end latency with a mean duration of 10–15 seconds (over 95% of the total) primarily due to LLM inference. In contrast, the **Near-RT RIC!** and E2 Node layers demonstrate sub-second latencies; the Orion xApp transformation completes in 70–100 ms including Hypertext Transfer Protocol (HTTP) discovery requests, while E2 Node processing takes approximately 1 ms due to efficient Abstract Syntax Notation One (ASN.1) decoding. While LLM-based interpretation introduces significant overhead, this latency is acceptable for network slice configuration timescales and leaves the critical RAN control path (under 300 ms) well within O-RAN Alliance requirements for non-real-time operations. Future optimizations may include intent caching or smaller, fine-tuned models to reduce inference time.

VII. CONCLUSION

ACKNOWLEDGMENT

This work has been partially funded by the project XGM-AFCCT-2024-5-1-1 supported by xGMobile - EMBRAPII -

Inatel Competence Center on 5G and 6G Networks, with financial resources from the PPI IoT/Manufatura 4.0 / the MCTI grant number 052/2023, signed with EMBRAPII.

REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, F. Restuccia, and T. Melodia, "Understanding open ran: Architecture, interfaces, and research challenges," *IEEE Communications Surveys & Tutorials*, 2023, provide volume/issue/pages.
- [2] H. et al., "Towards an ai/ml-driven smo framework in o-ran," 2024. [Online]. Available: <https://arxiv.org/abs/2409.05092>
- [3] L. Velasco, M. Signorelli, O. G. De Dios, C. Papagianni, R. Bifulco, J. J. V. Olmos, S. Pryor, G. Carrozzo, J. Schulz-Zander, M. Bennis, R. Martinez, F. Cugini, C. Salvadori, V. Lefebvre, L. Valcarenghi, and M. Ruiz, "End-to-end intent-based networking," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 106–112, 2021.
- [4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "RFC 9315: Intent-based networking-concepts and definitions," 2022.
- [5] 3GPP, "Ts 28.312 v18.8.0: Intent driven management services," 3GPP, Tech. Rep., 2025. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/128300_128399/128312/18.08.00_60/ts_128312v180800p.pdf
- [6] ETSI, "Gs zsm 016 v1.1.1: Intent-driven closed loops," ETSI, Tech. Rep., 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/ZSM/001_099/016/01.01.01_60/gs_ZSM016v010101p.pdf
- [7] T. et al., "Ina-infra: Open and extensible infrastructure for intent-driven network automation," 2024. [Online]. Available: <https://arxiv.org/abs/2410.09765>
- [8] M. A. Habib, H. Zhou, P. E. Iturria-Rivera, M. Elsayed, M. Bavand, R. Gaigalas, Y. Ozcan, and M. Erol-Kantarci, "Intent-driven Intelligent Control and Orchestration in O-RAN Via Hierarchical Reinforcement Learning," in *IEEE 20th International Conference on Mobile Ad Hoc and Smart Systems (MASS)*, 2023, pp. 55–61.
- [9] D. et al., "Towards end-to-end network intent management with llms," 2025. [Online]. Available: <https://arxiv.org/pdf/2504.13589v1>
- [10] NDSS FutureG'25 Authors, "LLM-xApp: Llm-powered xapp for intent-driven resource allocation in o-ran," in *NDSS FutureG Workshop*, 2025. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/futureg25-57.pdf>
- [11] S. Niknam and Others, "Intelligent o-ran for beyond 5g and 6g networks," *IEEE Network*, 2022, add volume/issue/pages.
- [12] A. Banerjee, S. S. Mwanje, and G. Carle, "An Intent-Driven Orchestration of Cognitive Autonomous Networks for RAN management," in *International Conference on Network and Service Management (CNSM)*, 2021, pp. 380–384.
- [13] C. V. Nahum, V. H. L. Lopes, R. M. Dreifuerst, P. Batista, I. Correa, K. V. Cardoso, A. Klautau, and R. W. Heath, "Intent-Aware Radio Resource Scheduling in a RAN Slicing Scenario Using Reinforcement Learning," *IEEE Transactions on Wireless Communications*, vol. 23, no. 3, pp. 2253–2267, 2024.
- [14] J. Zhang, H. Wei, D. Gao, N. Xia, D. Wang, S. Yan, and X. Liu, "Intent-Driven RAN Slice Orchestration: A Multi-Agent Deep Reinforcement Learning Based Approach," in *IEEE/CIC International Conference on Communications in China (ICCC Workshops)*, 2023, pp. 1–6.
- [15] K. Abbas et al., "IBNSlicing: intent-based network slicing framework for 5G networks using deep learning," in *Asia-Pacific Network Operations and Management Symposium*, 2020, pp. 19–24.
- [16] M. Xie, P. H. Gomes, J. Niemöller, and J. P. Waldemar, "Intent-driven management for multi-vertical end-to-end network slicing services," in *IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 1285–1291.
- [17] J. Mcnamara, D. Camps-Mur, M. Goodarzi, H. Frank, L. Chinchilla-Romero, F. Cañellas, A. Fernández-Fernández, and S. Yan, "NLP Powered Intent Based Network Management for Private 5G Networks," *IEEE Access*, vol. 11, pp. 36 642–36 657, 2023.
- [18] K. Mehmood, H. V. Kalpanie Mendis, K. Kravlevska, and P. E. Heegaard, "Intent-based network management and orchestration for smart distribution grids," in *2021 28th International Conference on Telecommunications (ICT)*, 2021, pp. 1–6.
- [19] ETSI, "Zero-touch network and Service Management (ZSM); Intent-driven autonomous networks; Generic aspects," European Telecommunications Standards Institute, Technical Report, 2024.
- [20] TM Forum, "Intent in autonomous networks v1.3.0," TM Forum, Technical Report, 2022.
- [21] —, "Intent management api tmf921 v5.0.0," TM Forum, 2024. [Online]. Available: <https://www.tmforum.org/oda/open-apis/directory/intent-management-api-TMF921/v5.0.0>
- [22] "Intent driven management," 3GPP Technology page, 2023. [Online]. Available: <https://www.3gpp.org/technologies/intent>
- [23] ETSI, "Ts 103 985 v1.1.0: A1 interface use cases and requirements," ETSI, Tech. Rep., 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/103900_103999/103985/01.01.00_60/ts_103985v010100p.pdf
- [24] L. et al., "LLM-hRIC: Llm-empowered hierarchical ric for o-ran," 2025. [Online]. Available: <https://arxiv.org/pdf/2504.18062>
- [25] D. la Cruz et al., "RSLAQ: Robust sla-driven ran qos with o-ran," 2025. [Online]. Available: <https://arxiv.org/pdf/2504.09187>
- [26] B. et al., "PACIFISTA: Characterizing and mitigating o-ran conflicts," 2024. [Online]. Available: <https://arxiv.org/pdf/2405.04395>
- [27] "O-ran alliance releases 60 new or updated technical documents since march 2025," O-RAN Alliance News, 2025. [Online]. Available: <https://www.o-ran.org/blog/o-ran-alliance-releases-60-new-or-updated-technical-documents-since-march-2025>
- [28] G. et al., "Adapshare: RL-based dynamic spectrum sharing for o-ran," 2024. [Online]. Available: <https://arxiv.org/abs/2408.16842>